DUNGEON 2: THE SMARTPHONE

COMPUTER SCIENCE DUNGEON ADVENTURES

pencil · paper · one die

Your first crawl went so well that you have a new job: crawl **inside a smartphone**. Its owner says it has been acting strangely. The lock screen keeps looping, the camera roll is scrambled, and the photo app sorts pictures all wrong. Crawl five floors of the phone, index its **DATA**, and keep your **RAM** up. At the bottom, the photo app's AI is waiting to learn something new, and this time **you** are its teacher.

DRAW YOUR CRAWLER

CRAWLER NAME _____

REAL NAME _____

CLASS _____

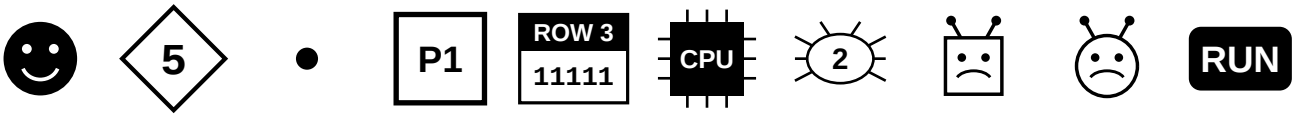
DATE STARTED _____

DATE FINISHED _____

YOU START WITH

10	0	10
RAM	**DATA**	**MAX RAM**

RAM is your energy. Broken code drains it.
DATA is what you index. Spend it in the shop.
MAX RAM is the most RAM you can hold.



HOW TO PLAY

Goal: get through all 5 floors. Collect DATA. Do not run out of RAM. **You need:** this booklet, a pencil and one die.

YOUR TURN

1. Roll the die.

2. Odd or even?
ODD (1, 3, 5): move **diagonally**.
EVEN (2, 4, 6): move **straight** (up, down, left or right).

3. Pick a direction and move that many squares, one square at a time.

4. Draw your path as you go. Mark the square where you stop with an **X**. Start there next turn.

5. Use everything you cross. Every bug, data dot and power-up on your path happens to you, in order. Write each gain or loss in the boxes at the bottom of the page, then cross the thing out. It only works once.

MOVING RULES

• If any direction lets you make your **whole move without hitting a wall**, you must pick one of those directions.

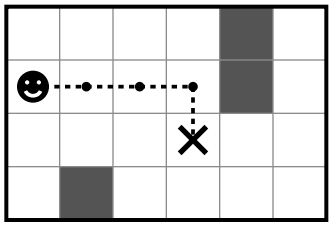
• **Keep going the same way.** You turn only when a wall or the edge of the map blocks you. Then pick a new direction of the same kind and keep counting.

• You may **not** turn to dodge a bug or to grab something.

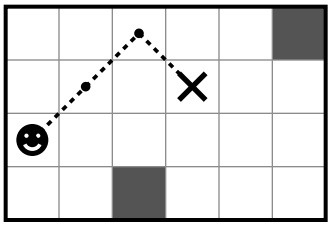
• Go back the way you came **only** if there is no other way.

• Moving diagonally, only the square you land on matters. You can slip between two wall corners.

EXAMPLES



Rolled 4 (even). Moved right 3 squares. A wall blocked the way, so turned and finished the last square going down.



Rolled 3 (odd). Moved up-right 2 squares. The edge blocked the way, so bounced down-right for the last square.

LEAVING A FLOOR

Stop on the **RUN** square, stop on any square touching it (corners count), or pass over it. Then you may leave. You can also stay to collect more.

Now do the math, once for RAM and once for DATA:
START + gained – lost = END

Copy both END numbers to START on the next floor.

RAM cannot go above your Max RAM. DATA cannot go below 0. You do not crash in the middle of a floor: only the END number counts.

CRASH!

If your END RAM is **0 or less**, your crawler crashed. Tally one crash on the last page. Start the next floor with **10 RAM and 0 DATA**. Keep playing!

CODE REVIEW

Real coders check each other's work. After every floor, swap booklets with a partner. Check their path, their math and their floor puzzle. Found a mistake? Fix it together.

FLOOR 1 THE LOCK SCREEN

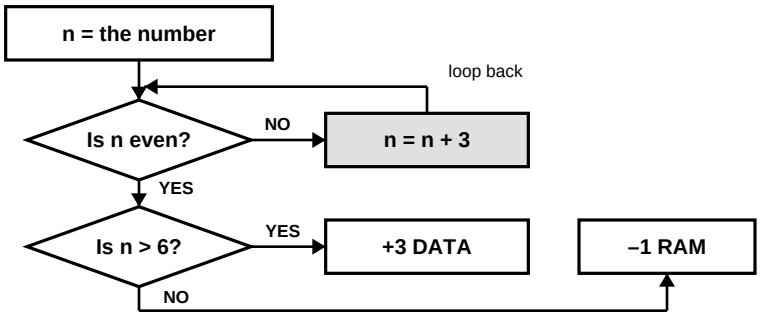
Algorithms: flowcharts, decisions and loops

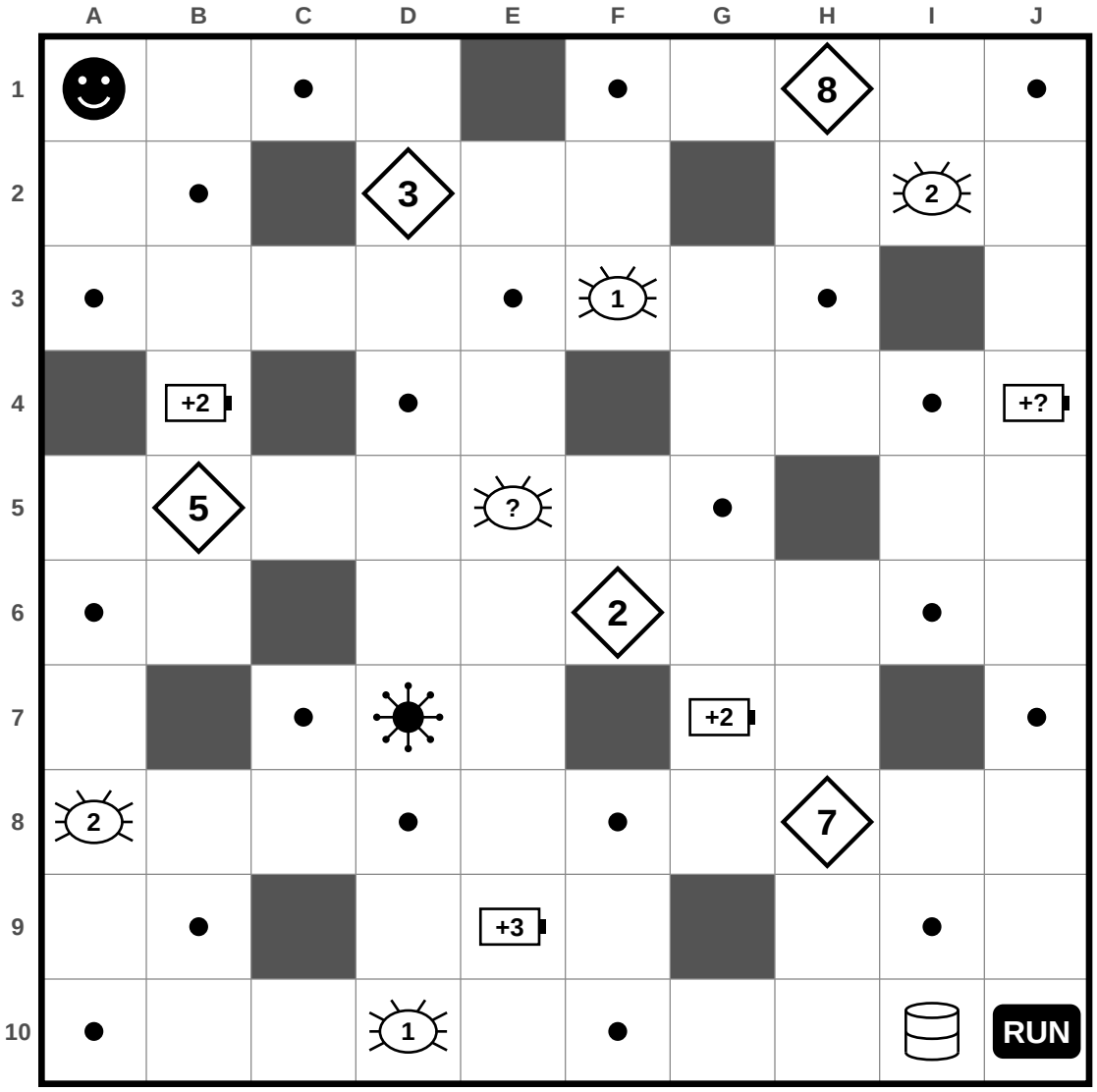
FLOOR RULE: FLOWCHART SQUARES

An **algorithm** can make decisions. A **flowchart** draws one: boxes are steps, diamonds are questions. Cross a flowchart square and run the flowchart with its number as **n**. Follow the arrows until you reach a RAM or DATA box. Keep moving after.

Write every value n takes, so your partner can check it.
MY TRACES (example: 3 → 6)

Q ___ : _____ Q ___ : _____
Q ___ : _____ Q ___ : _____
Q ___ : _____ Q ___ : _____





KEY

- **Start**
- **Exit.** Stop on it, next to it, or pass over it
- **Data:** +1 DATA
- **Database:** roll, gain that much DATA
- **Bug:** lose that much RAM
- **Mystery bug:** roll, lose that much RAM
- **Power-up:** gain that much RAM
- **Mystery power-up:** roll, gain that RAM
- **Malware:** STOP, even in a program. Roll, lose that DATA
- **Flowchart:** run the flowchart with this number

	START	+ GAINED	– LOST	END
RAM				
DATA				

END = START + gained – lost. RAM cannot go above your Max RAM. DATA cannot go below 0. END RAM of 0 or less = CRASH.

FLOOR 2 THE APP STORE

Programming: loops that repeat until something happens

FLOOR RULE: REPEAT UNTIL

Step on a program square and your move stops: the rest of your roll is lost. Run the program.

REPEAT [...] UNTIL WALL is a loop that does not know how many times it will run. Before each repeat, look ahead: if any step in the brackets would hit a wall or the edge, **the loop ends**. It might even run 0 times.

Count each repeat: **+1 DATA per repeat**. **Malware stops a program** where it is. Each program runs once.

THE PROGRAMS

P1 `REPEAT [ → ] UNTIL WALL`

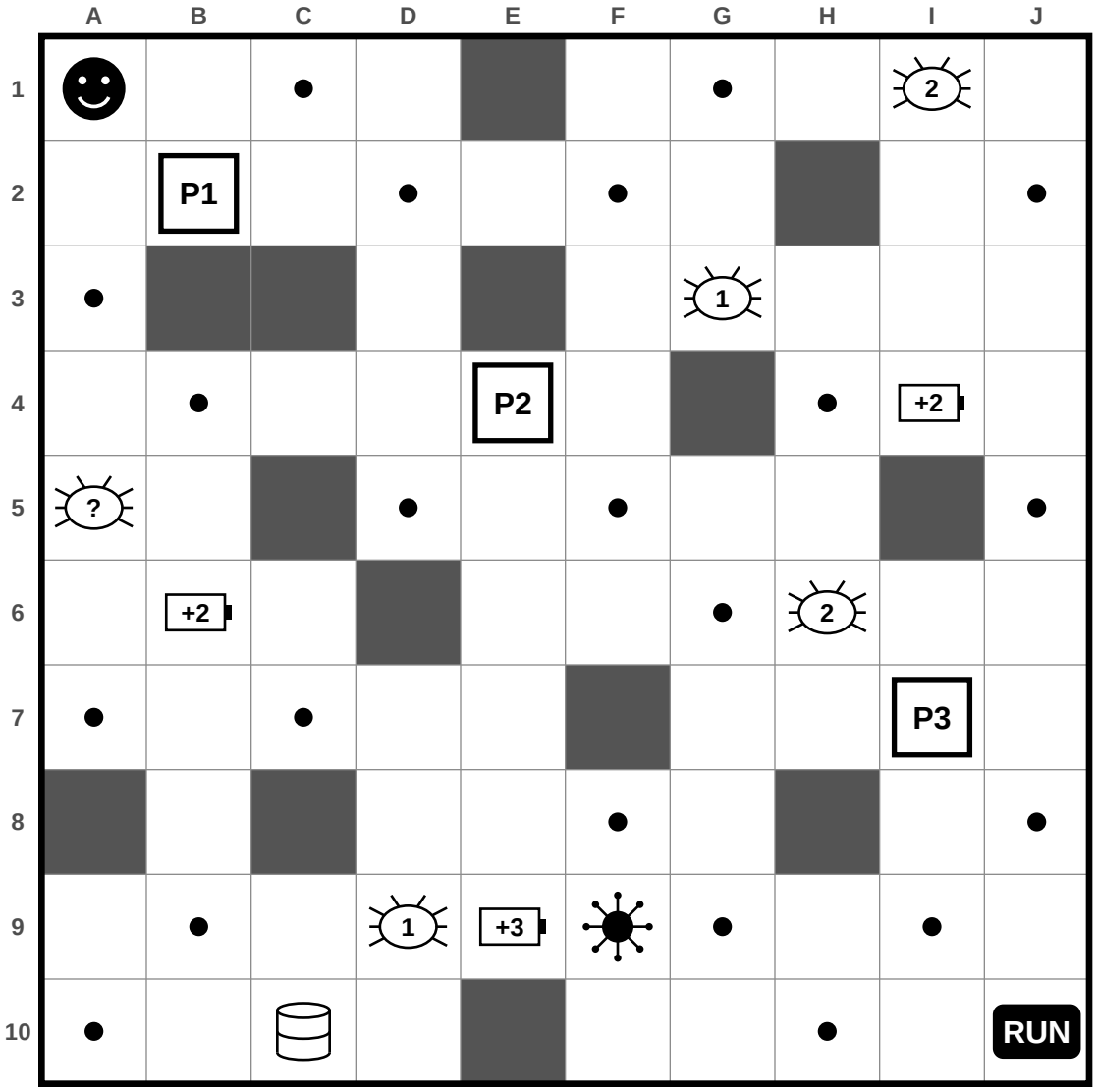
Ended on: _____ Loop ran: _____ times

P2 `REPEAT [ ↓ → ] UNTIL WALL`

Ended on: _____ Loop ran: _____ times

P3 `↓ ↓ REPEAT [ ← ] UNTIL WALL`

Ended on: _____ Loop ran: _____ times



KEY

- **Start**
- **Exit.** Stop on it, next to it, or pass over it
- **Data:** +1 DATA
- **Database:** roll, gain that much DATA
- **Bug:** lose that much RAM
- **Mystery bug:** roll, lose that much RAM
- **Power-up:** gain that much RAM
- **Mystery power-up:** roll, gain that RAM
- **Malware:** STOP, even in a program. Roll, lose that DATA
- **Program:** stop and run it. Count the loops

	START	+ GAINED	– LOST	END
RAM				
DATA				

END = START + gained – lost. RAM cannot go above your Max RAM. DATA cannot go below 0. END RAM of 0 or less = CRASH.

FLOOR 3 THE CAMERA ROLL

Data: pictures stored as pixels and bits

FLOOR RULE: PICTURES ARE NUMBERS

A photo is a grid of tiny squares called **pixels**. In a black-and-white image each pixel is one **bit**: **1** = black, **0** = white.

This phone's photo is broken into rows and hidden in image chests. Open a chest: +1 DATA. Copy its row into the grid, shading every 1.

Find all 5 rows: **+5 DATA**. Your partner checks the picture.

THE PHOTO

1					
2					
3					
4					
5					

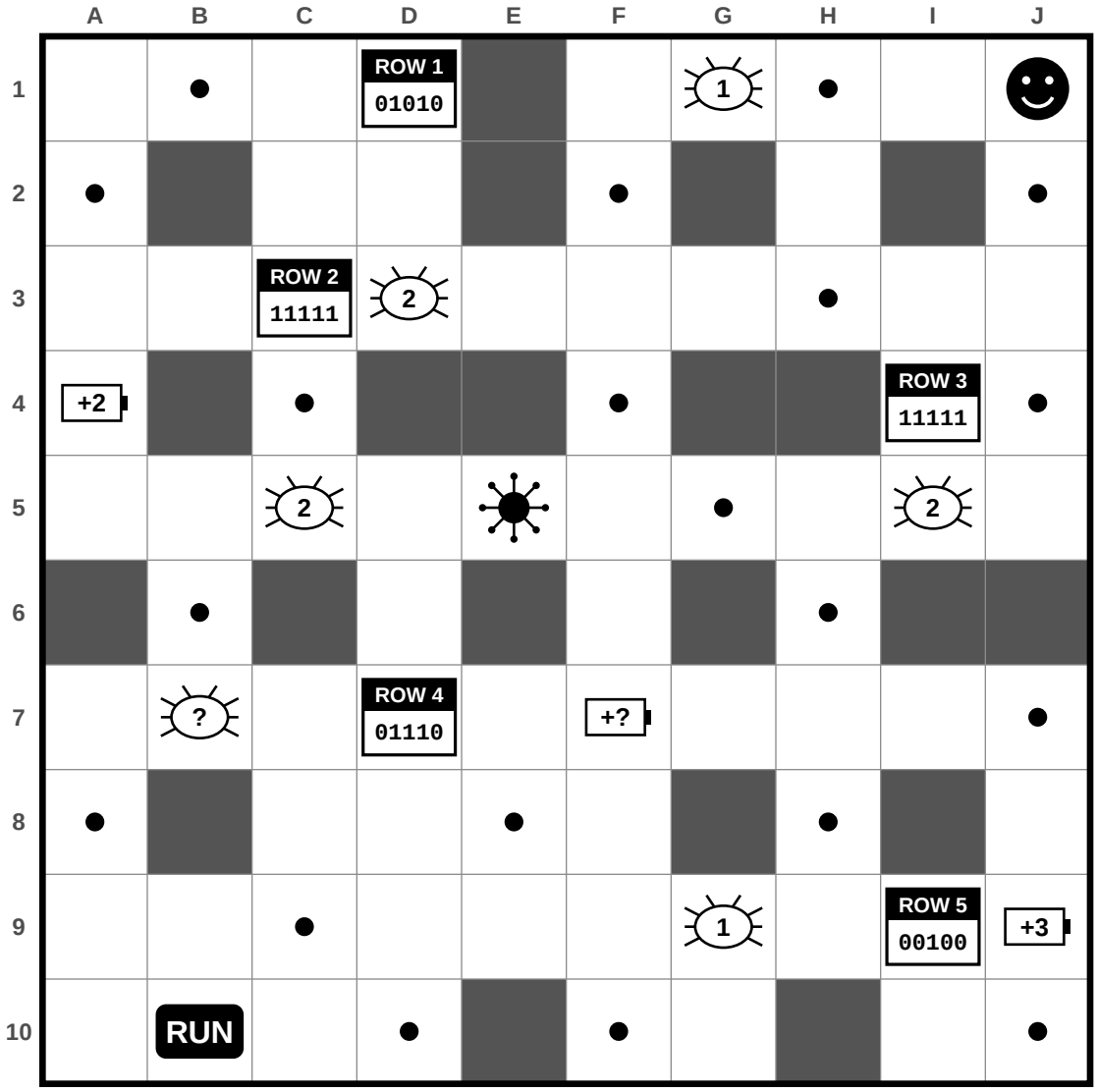
It is a picture of:

_____

Pixels in the photo: ____

Bits to store it: ____

Bits, 10 × 10 photo: ____



KEY

- **Start**
- **Exit.** Stop on it, next to it, or pass over it
- **Data:** +1 DATA
- **Database:** roll, gain that much DATA
- **Bug:** lose that much RAM
- **Mystery bug:** roll, lose that much RAM
- **Power-up:** gain that much RAM
- **Mystery power-up:** roll, gain that RAM
- **Malware:** STOP, even in a program. Roll, lose that DATA
- **Image chest:** +1 DATA. Copy its row of pixels

	START	+ GAINED	– LOST	END
RAM				
DATA				

END = START + gained – lost. RAM cannot go above your Max RAM. DATA cannot go below 0. END RAM of 0 or less = CRASH.

FLOOR 4 THE BATTERY AND CHIP

Systems: input, processing, storage and output

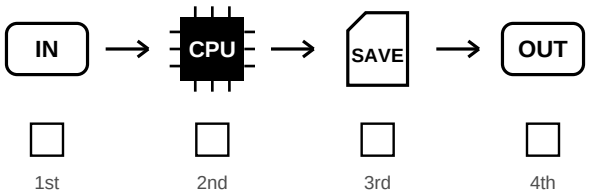
FLOOR RULE: FOLLOW THE SIGNAL

Every computer does four jobs, always in this order: it takes **input**, **processes** it, **stores** the result, then shows the **output**.

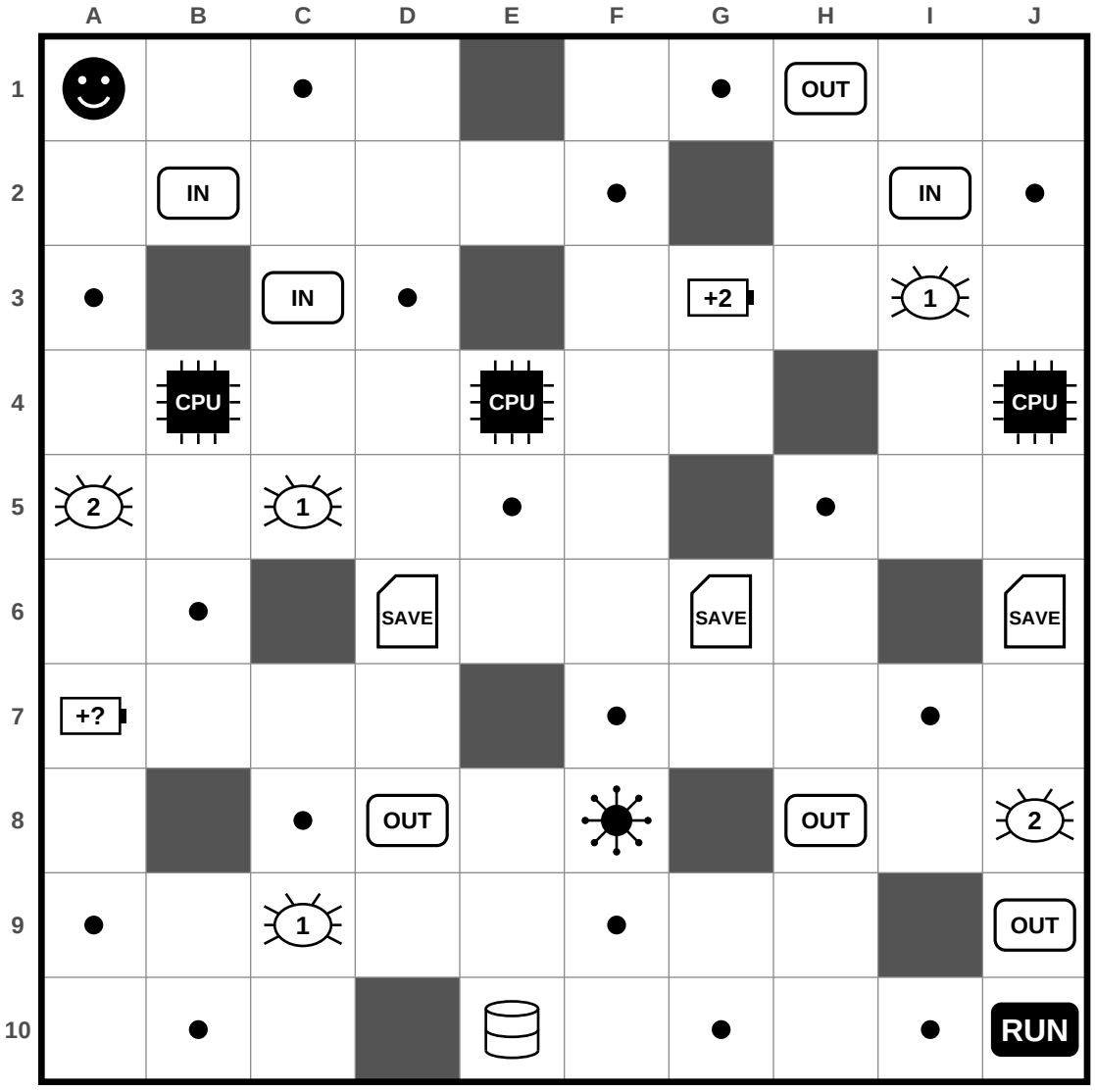
Carry a signal through the phone. Cross the part that is **next** in the order, or stop touching it (corners count): tick it, **+1 DATA**, cross it out. A part out of order does nothing.

The RUN square stays shut until all four are ticked.

THE SIGNAL



All four ticked: the RUN square opens.



KEY

- **Start**
- **Exit.** Stop on it, next to it, or pass over it
- **Data:** +1 DATA
- **Database:** roll, gain that much DATA
- **Bug:** lose that much RAM
- **Mystery bug:** roll, lose that much RAM
- **Power-up:** gain that much RAM
- **Mystery power-up:** roll, gain that RAM
- **Malware:** STOP, even in a program. Roll, lose that DATA
- **1. IN:** touchscreen, microphone
- **2. CPU:** the processor follows instructions
- **3. SAVE:** storage keeps it
- **4. OUT:** screen, speaker

	START	+ GAINED	– LOST	END
RAM				
DATA				

END = START + gained – lost. RAM cannot go above your Max RAM. DATA cannot go below 0. END RAM of 0 or less = CRASH.

FLOOR 5 THE PHOTO SORTER

AI: how an AI learns from the examples it is given

FLOOR RULE: TRAIN THE AI

The phone's photo sorter is an AI that **learns from you**. When you meet a bot, check its face. **Smiling** = friendly, +1 RAM. **Frowning** = hostile, –1 RAM. Then add a tally for its **body shape**: that tally is your training data.

When you leave, the AI learns: for each shape, the label with more tallies wins. A tie teaches it nothing. Then it sorts the test bots: **+1 DATA** for each one it gets right.

TRAINING DATA

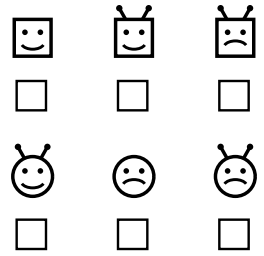
	FRIENDLY	HOSTILE
SQUARE		
ROUND		

THE AI LEARNED

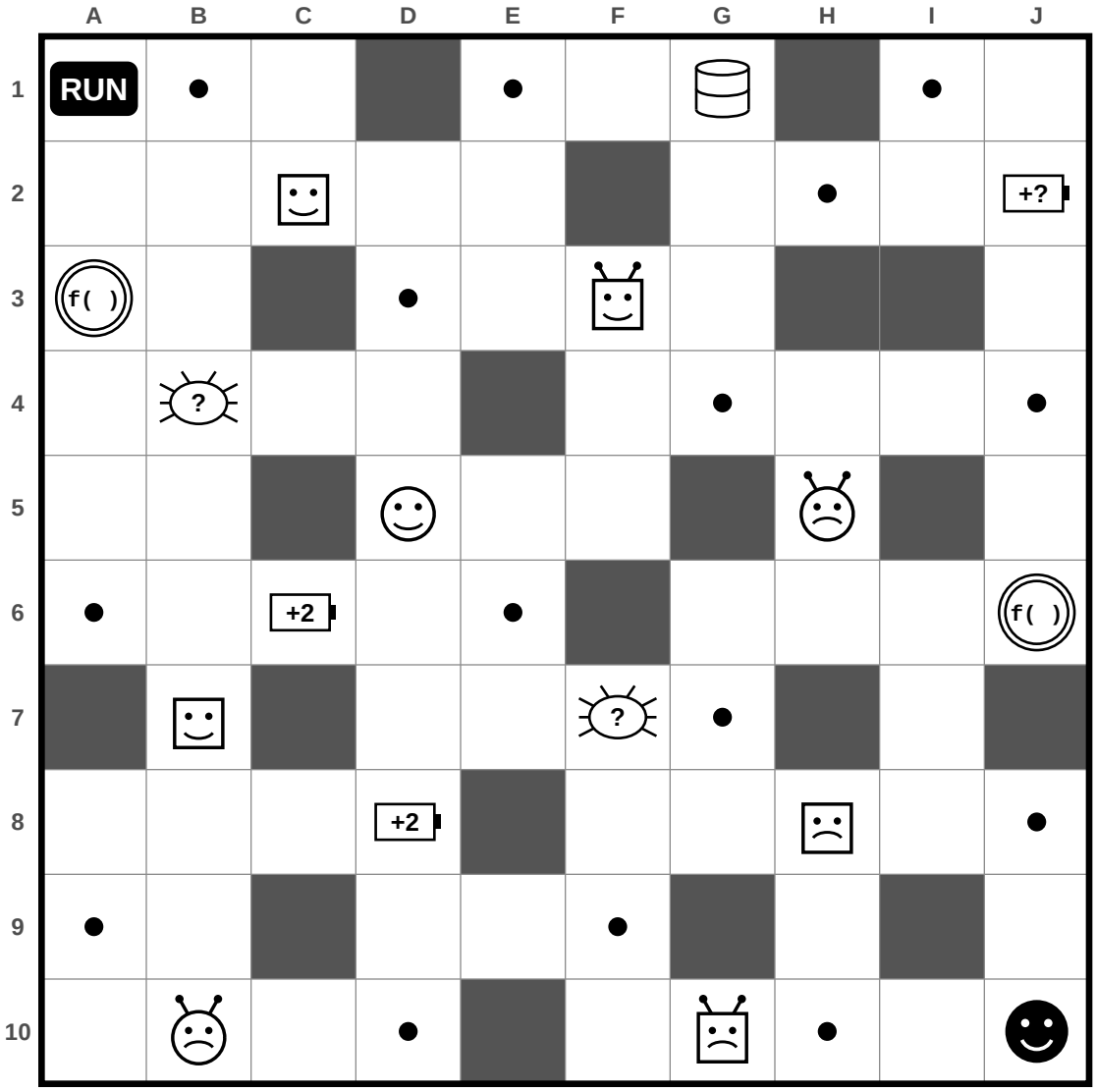
Square bots are: _____

Round bots are: _____

TEST BOTS



Tick each one the AI got right



KEY

- **Start**
- **Exit.** Stop on it, next to it, or pass over it
- **Data:** +1 DATA
- **Database:** roll, gain that much DATA
- **Bug:** lose that much RAM
- **Mystery bug:** roll, lose that much RAM
- **Power-up:** gain that much RAM
- **Mystery power-up:** roll, gain that RAM
- **Malware:** STOP, even in a program. Roll, lose that DATA
- **Function:** jump to the other f( ) every time, keep going
- **Bot:** label it from its face, tally it

	START	+ GAINED	– LOST	END
RAM				
DATA				

END = START + gained – lost. RAM cannot go above your Max RAM. DATA cannot go below 0. END RAM of 0 or less = CRASH.

THE SHOP

Visit the shop twice: **after Floor 2** and **after Floor 4**. Pay with DATA. Write the price in the **– LOST** box for DATA on the next floor. You can buy each thing once per visit.

ITEM	PRICE	WHAT IT DOES	SHOP 1	SHOP 2
CHARGER	4 DATA	Gain 4 RAM. Write +4 in the + GAINED box for RAM on the next floor.	☐ bought	☐ bought
UNDO	3 DATA	Roll again once, on any roll you do not like.	☐ bought ☐ used	☐ bought ☐ used
ANTIVIRUS	5 DATA	Ignore one bug or one malware. Cross it out without losing anything.	☐ bought ☐ used	☐ bought ☐ used
MEMORY CARD	8 DATA	Your Max RAM goes up by 2 for the rest of the game.	☐ bought	☐ bought

MY MAX RAM: **10** → ☐ → ☐

FINAL STATS

CRASHES (tally)	FINAL RAM	FINAL DATA	FLOORS CLEARED

YOUR RANK (by final DATA)

- ☐ **App Tester** — 0 – 14 DATA
- ☐ **App Developer** — 15 – 29 DATA
- ☐ **Mobile Engineer** — 30 – 44 DATA
- ☐ **Chief Technology Officer** — 45 + DATA

THINK BACK

Which floor puzzle was hardest? What made it hard?

_____

_____

On Floor 5, did your AI learn the right thing? What training data would have helped?

_____

_____

Name one thing your real phone does that uses an idea from this booklet.

_____

_____

TEACHER PAGE: ANSWER KEY (do not hand out)

FLOOR 1: flowchart
H1: 8 = **+3 DATA**
D2: 3 → 6 = **–1 RAM**
B5: 5 → 8 = **+3 DATA**
F6: 2 = **–1 RAM**
H8: 7 → 10 = **+3 DATA**

FLOOR 2: REPEAT UNTIL loops
P1 ends on **G2**; the loop runs 5 times (+5 DATA)
P2 ends on **H7**; the loop runs 3 times (+3 DATA)
P3 ends on **F9**; the loop runs 3 times (+3 DATA). Malware stops it.
If that malware is already crossed out: ends on **A9**, 8 times

FLOOR 3: pixels
The picture is **a heart**: row 1 01010, row 2 11111, row 3 11111, row 4 01110, row 5 00100. 25 pixels, 25 bits (one per pixel); a 10 × 10 photo needs 100 bits.

FLOOR 4: hardware
IN at B2, I2, C3; CPU at B4, E4, J4; SAVE at D6, G6, J6; OUT at H1, D8, H8, J9. Parts crossed out of order stay on the map. A finished signal earns +4 DATA; the RUN square is shut until then.

FLOOR 5: training data
C2: square, smiling (friendly)
F3: square, smiling (friendly)
D5: round, smiling (friendly)
H5: round, frowning (hostile)
B7: square, smiling (friendly)
H8: square, frowning (hostile)
B10: round, frowning (hostile)
G10: square, frowning (hostile)
Meeting every bot teaches square = friendly, round = hostile, **4 of 6** test bots right. Looking only at shape, no AI can get more than 4.
Meeting only the two bots nearest the start (H8, G10) teaches square = hostile, round = nothing, **1 of 6** test bots right: a skewed sample.

WHAT EACH FLOOR TEACHES
1. Algorithms and Design: flowcharts, decisions and a loop inside an algorithm, tracing.
2. Programming: indefinite loops (REPEAT UNTIL), loop conditions, counting iterations.
3. Data and Analysis: images as data, pixels and bits.
4. Systems and Security: the input, processing, storage and output model of a computer.
5. Computing and Society (AI): machine learning from labelled training data, and how a skewed sample produces a skewed model.
Builds on Game 1 (loops, binary, the sorting AI) but does not require it. Match these to your state or CSTA standard codes before use; the codes are not listed here.

RUNNING IT
• Each student needs a booklet, a pencil and a die. Print single-sided.
• If students have not played Game 1, teach the movement rules on the board first.
• Floor 4's RUN square stays shut until the signal is finished, so this floor runs longest.
• Real classroom timing is not yet tested: time your first session.
• Code review pairs replace teacher marking. Keep this page for disputes.

DISCUSSION STARTERS
1. Could you draw the flowchart as a program with IF and REPEAT?
2. When is a REPEAT UNTIL loop better than REPEAT 3? Can a loop run forever?
3. How many bits does a real 12-megapixel colour photo need?
4. Name an input, processor, storage and output in a games console or a car.
5. Compare AIs across the class. Why did they learn different rules from the same floor?